



KIET
GROUP OF INSTITUTIONS

Connecting Life with Learning



A

Project Synopsis

on

District-Level Healthcare Monitoring Portal

submitted as partial fulfillment for the award of

**BACHELOR OF TECHNOLOGY
DEGREE**

SESSION 2023-27

in

Computer Science & Engineering

By

Priyansh Singh(2300290100190) (Team Leader)

Dev Sharma (2300290100099)

Dheeraj Yadav (2300290100101)

Prashant Kumar (2300290100180)

Under the supervision of

Mr. Pavan Sharma

KIET Group of Institutions, Ghaziabad

Affiliated to

Dr. A.P.J. Abdul Kalam Technical University, Lucknow
(Formerly UPTU)

Academic Year 2025-26

Abstract

The District-Level Hospital Monitoring Portal is a web-based, centralized information system developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The portal's main objective is to provide district health administrators and hospital managers with a single-pane view of healthcare delivery indicators—patient inflow, disease incidence, resource availability (beds, oxygen, medicines, ventilators), and staff allocation—across all hospitals within a district. By aggregating structured reports and live updates from participating hospitals, the system enables near real-time situational awareness that is crucial during normal operations and public health emergencies.

A notable feature of the platform is a lightweight AI-driven analytics module that performs anomaly detection on time-series case data and resource usage. The module flags unusual spikes or clusters of reported cases, which can indicate outbreaks or data quality issues needing human review. Alerts are surfaced via the administrative dashboard and can be configured to send email/SMS notifications to designated officials. The portal also retains historical records to support trend analysis and policy making.

This synopsis outlines the motivation for the project (fragmented district-level reporting and slow response to emerging health threats), the system architecture, the planned methodology for development and testing, required facilities, and references. The expected outcomes include improved transparency in healthcare reporting, optimized resource distribution during crises, and earlier detection of localized disease surges—thereby strengthening district-level public health response capabilities.

Introduction

District-level healthcare administration depends heavily on timely, accurate, and consolidated information from all hospitals operating within a region. However, in most districts, hospitals still maintain independent reporting systems, relying on manual spreadsheets, delayed email submissions, or fragmented software tools that are not interconnected. This creates significant challenges for public health officials who must monitor disease trends, allocate critical medical

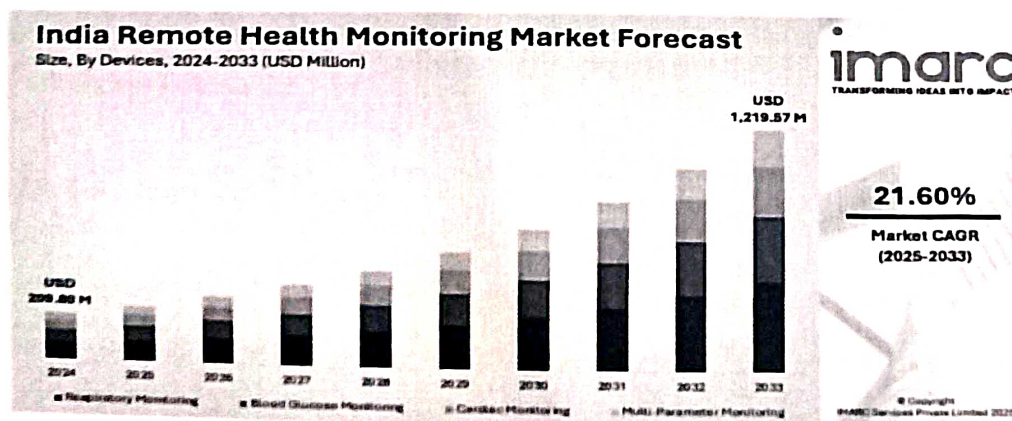


Figure 1 Indian Remote Health Monitoring Market Forecast

resources, identify emerging outbreaks, and coordinate emergency responses. The lack of a unified digital ecosystem results in delayed decision-making, inefficient resource utilization, and limited visibility into real-time healthcare conditions across the district.

The **District-Level Hospital Monitoring Portal** aims to solve these issues by providing a centralized web-based platform built using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**. The portal serves as a single, integrated dashboard that aggregates key operational indicators from multiple hospitals—patient inflow, disease incidence, bed occupancy, ICU and ventilator availability, medicine stock levels, and staff capacity. By establishing a standardized data submission model, the system ensures that all hospitals report using a uniform structure, making district-wide comparisons and trend analyses more reliable and meaningful.

A major gap in existing healthcare systems is the absence of real-time surveillance and automated early-warning mechanisms. To address this, the proposed portal incorporates a lightweight **AI-driven anomaly detection module** capable of identifying unusual spikes in disease cases or sudden drops in resource availability. Such automated alerts can help health authorities initiate timely interventions, conduct rapid assessments, and prevent small clusters from evolving into major outbreaks. Additionally, historical data stored in the portal supports long-term planning, policy evaluation, and predictive analytics for future health emergencies.

This project contributes to the field of **health informatics** by integrating real-time data pipelines, analytics, and visualization into one cohesive system. It emphasizes usability through an intuitive React-based interface and ensures scalability through a modular backend architecture. Ultimately, the portal enhances healthcare transparency, strengthens district-level coordination, and supports data-driven decision-making—paving the way for a more responsive and efficient public health ecosystem.

Literature Survey

Overview of Related Systems:

Several national and regional health information systems (e.g., government HMIS deployments, eHospital implementations) provide structured record keeping and centralized storage of patient and administrative data. Many academic papers and pilot projects discuss dashboards and analytical tools for hospital management. A common observation in literature is that while patient-level EHRs and hospital administrative systems work well within organizational boundaries, cross-hospital district-level analytics are less mature.

Survey of Key Studies and Systems:

- 1. Centralized HMIS implementations:** These systems capture routine facility reports and have been successful in generating national statistics but often lack real-time inter-hospital visibility at the district level.

2. Dashboard-focused research: Multiple studies propose visual analytics for hospital operations—bed capacity monitoring, ICU occupancy tracking, and supply chain dashboards. These works emphasize the need for intuitive visualizations and alerting to improve operational decisions.

3. AI in public health surveillance: Research on using anomaly detection and clustering techniques for outbreak detection shows promising results in early warning when data pipelines are reliable. Authors recommend hybrid approaches (statistical rules + machine learning) to reduce false positives.

Gap Analysis:

While the literature provides strong foundations in dashboard design and algorithmic surveillance, fewer projects demonstrate a complete, end-to-end district-level portal that integrates heterogeneous hospital data, delivers real-time alerts, and supports role-based workflows. The proposed project aims to implement a practical, deployable prototype addressing those gaps.

Methodology/ Planning of work

Development Phases:

Phase 1 — Requirements and Design (Weeks 1-3):

- Stakeholder interviews and requirement consolidation.
- Define data model and standard reporting fields.
- High-level architecture diagram and API contracts.

Phase 2 — Backend and Database (Weeks 4-7):

- Implement MongoDB schema and validation rules.
- Develop RESTful APIs and WebSocket endpoints using Node.js and Express.js.
- Authentication, RBAC implementation and audit logging.

Phase 3 — Frontend and Dashboard (Weeks 8-11):

- Build React components for data entry, dashboards, and charts.
- Integrate real-time updates and filtering options.
- Implement configurable alert settings and notification channels.

Phase 4 — Analytics and Alerts (Weeks 12-14):

- Implement statistical anomaly detection (moving averages, z-scores).
- Prototype simple ML model if sufficient historical data is available.
- Create alert workflows and escalation paths.

Phase 5 — Testing, Deployment and Documentation (Weeks 15-18):

- Unit and integration testing, performance testing for concurrent updates.
- Usability testing with sample hospital users and iterative fixes.
- Deployment on a cloud instance (or on premise server) and final documentation.

Facilities required for proposed work

Software Requirements:

- MongoDB (community edition) — for document-oriented storage and flexible schema management.
- Node.js and Express.js — backend server and API framework.
- React.js — frontend framework for building single-page application dashboards.
- Charting library (e.g., Recharts or Chart.js) — for interactive visualizations.
- Basic ML/Statistics libraries (scikit-learn for prototyping or TensorFlow.js for lightweight models).
- Version control (Git) and collaboration tools (GitHub or GitLab).
- Postman for API testing, and VS Code or similar IDE.

Hardware Requirements:

- Development machines (Laptop/PC) with at least 8GB RAM, recommended 16GB for comfortable development.
- A test server or cloud instance (e.g., AWS/Azure/GCP or local VM) for deployment and integration testing.
- Stable broadband internet connectivity for collaborating and pulling dependencies.

Human Resources:

- Project team (4 students): frontend developer, backend developer, data/analytics developer, and tester/documentation lead.
- A faculty supervisor for technical guidance and evaluation.

Other Facilities:

- Access to anonymized historical hospital reporting data (if available) for training and validation of analytics.
- Lab space for demonstration and final presentations.

References

- [1] M. MongoDB, "MongoDB Manual and Documentation," MongoDB Inc., 2022.
- [2] R. Smith and A. Kumar, "Dashboard-based hospital monitoring systems," *International Journal of Health Informatics*, vol. 10, no. 2, pp. 45-58, 2020.
- [3] Government Health Management Information Systems (HMIS) - implementation guides.
- [4] J. Lee et al., "Anomaly detection for public health surveillance," *Proc. of IEEE HealthCom*, pp. 112-120, 2019.
- [5] MERN Stack tutorials and best practices, assorted online resources.